InfraGuid

Enterprise AI Operations Copilot

Security Architecture &

Cryptographic Controls

Classification:	**CONFIDENTIAL — INTERNAL USE ONLY**
Document Version:	1.0
Last Reviewed:	June 2026
Audience:	Security Engineering, Platform Architects, Compliance

InfraGuid serves as the centralized Enterprise AI Operations Copilot for organizational knowledge retrieval, operational guidance, incident investigation, infrastructure automation, and DevOps decision support.

Unlike traditional chatbot platforms, InfraGuid processes and retrieves sensitive organizational information — including architecture standards, deployment runbooks, incident response procedures, security guidelines,

historical outage reports, and operational best practices. Because the platform becomes a central repository for institutional knowledge, protecting the confidentiality, integrity, and availability of stored information is a foundational architectural requirement.

Core Security Principles

- Cryptographic protection of all sensitive data at rest and in transit.
- Elimination of hardcoded credentials through dynamic secret retrieval.
- Strict identity-based access control via AWS IAM least-privilege roles.
- Complete auditability of all privileged operations via CloudTrail.

Platform Components

- AWS Key Management Service (KMS) — Cryptographic root of trust
- AWS Secrets Manager — Zero-hardcoded-secrets secret storage
- AWS IAM — Identity and access governance
- Amazon S3 — Encrypted organizational knowledge repository
- Amazon DynamoDB — Encrypted session and metadata storage
- Amazon EFS — Encrypted vector storage for ChromaDB
- Amazon Bedrock — Managed LLM inference and embeddings
- ChromaDB — Vector store for semantic knowledge retrieval

2.1 Protection of Organizational Knowledge

The platform stores documents containing internal operational knowledge. Exposure of these documents could reveal internal infrastructure patterns, security practices, and operational procedures.

Protected Document Categories

- Architecture Standards
- Security Standards and Policies
- Deployment Procedures and Runbooks
- Incident Response SOPs
- Production Incident Reports and RCA Documents
- Infrastructure Governance Documents

All stored documents are encrypted at rest using AWS KMS Customer Managed Keys and remain inaccessible from public networks through enforced VPC and bucket-level access controls.

2.2 Protection of Application Secrets

The platform requires access to various sensitive configuration values. These must never exist within source code repositories, Docker images, Terraform variable files, EC2 user data, or application logs. Instead, secrets are retrieved dynamically at runtime through AWS Secrets Manager.

Protected Secret Categories

- MongoDB connection strings
- JWT signing secrets
- Internal API keys
- ChromaDB configuration
- Administrative credentials

2.3 Cryptographic Separation of Duties

The architecture intentionally separates data storage, secret storage, and cryptographic key management. This ensures that compromise of a single service does not automatically grant access to encrypted information.

Separation Guarantees

- Compromise of an S3 bucket does not provide access to encryption keys.
- Compromise of an EC2 instance does not grant administrative control over KMS.
- Compromise of a database does not expose application secrets.

This separation significantly reduces the blast radius during security incidents.

The diagram below illustrates the full security architecture of the InfraGuid platform — spanning the client layer, network delivery, application services, encrypted data stores, the KMS security control plane, and the audit monitoring layer.

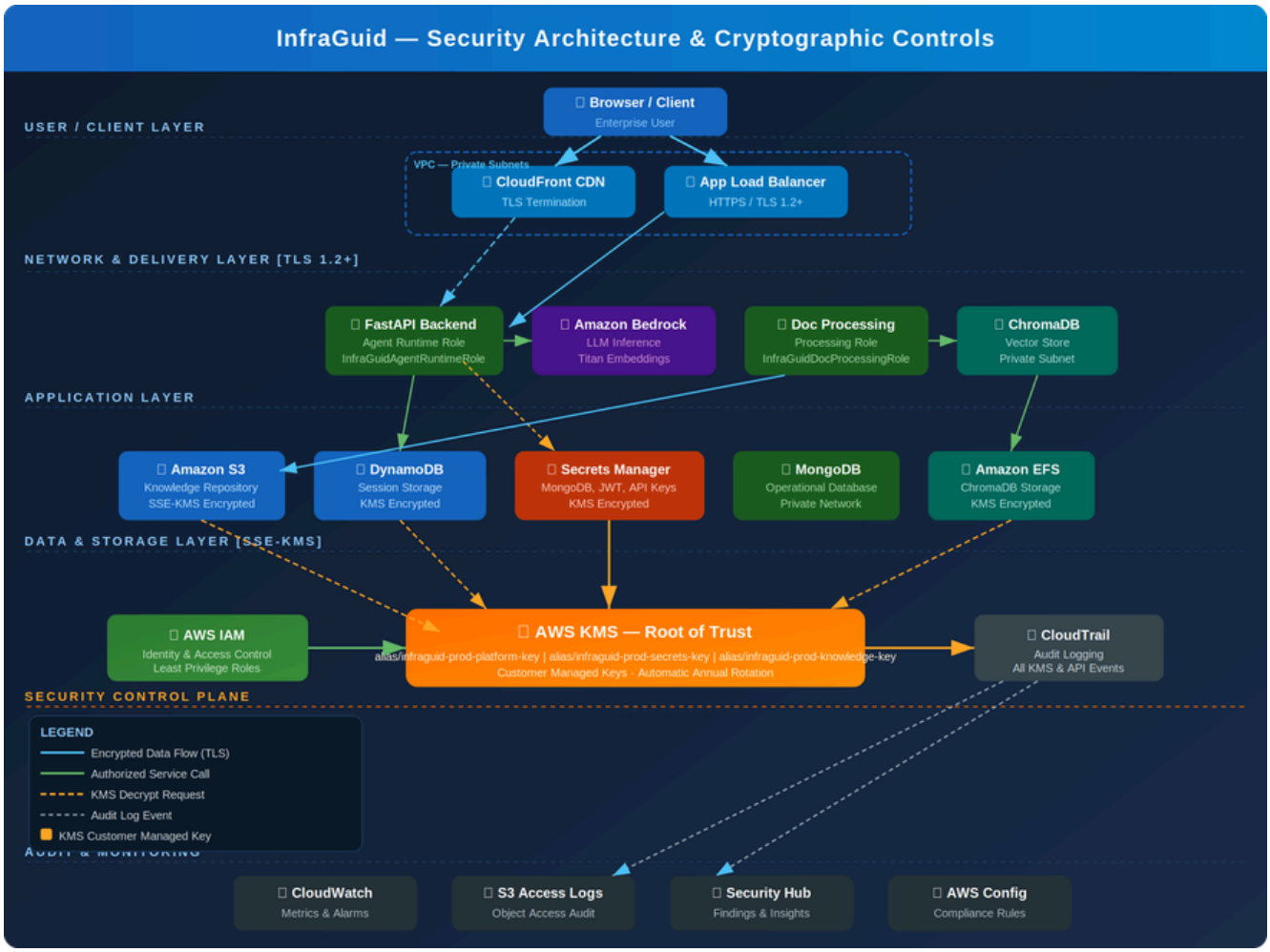

Figure 1 — InfraGuid Security Architecture & Cryptographic Controls

InfraGuid implements encryption across three independent layers to ensure data remains protected at all stages of its lifecycle.

4.1 Layer 1 — Data at Rest Encryption

All persistent storage services are encrypted using AWS KMS Customer Managed Keys.

Amazon S3	SSE-KMS encryption using alias/infraguid-prod-knowledge-key

Amazon DynamoDB	Encryption using alias/infraguid-prod-platform-key

Amazon EFS	KMS-encrypted filesystem backing ChromaDB vector storage

Secrets Manager	Encrypted using alias/infraguid-prod-secrets-key

4.2 Layer 2 — Data in Transit Encryption

All communication channels enforce TLS 1.2 or higher. Unencrypted communication is prohibited.

- Browser → CloudFront (TLS 1.2+)
- CloudFront → Application Load Balancer (HTTPS)
- ALB → FastAPI Backend (HTTPS)
- Backend → Amazon Bedrock (TLS)
- Backend → DynamoDB (TLS)
- Backend → Secrets Manager (TLS)

4.3 Layer 3 — Application-Level Access Control

Even if encrypted resources are successfully accessed, decryption operations require IAM authorization, KMS authorization, and resource-level authorization. Multiple authorization boundaries must be satisfied before sensitive data becomes accessible, creating a layered security model rather than a single point of protection.

InfraGuid uses Customer Managed Keys (CMKs) rather than AWS-managed keys. This decision achieves fine-grained access control, dedicated audit trails, independent key rotation, explicit cryptographic ownership, and future compliance readiness.

5.1 KMS Key Hierarchy

Key Alias	Purpose	Rotation
alias/infraguid-prod-platform-key	DynamoDB encryption, application metadata, general service encryption	Annual Auto
alias/infraguid-prod-secrets-key	Secrets Manager encryption — MongoDB, JWT, API keys, admin credentials. Only runtime role may decrypt.	Annual Auto
alias/infraguid-prod-knowledge-key	S3 knowledge repository — architectural standards, runbooks, incident reports, governance docs	Annual Auto

By separating keys according to business function, the platform reduces the impact of accidental permission grants and enables future cryptographic segmentation.

5.2 Cryptographic Trust Model

The InfraGuid platform follows a centralized cryptographic trust model in which AWS KMS acts as the root of trust for all encryption operations. All encryption and decryption activities are delegated to KMS, ensuring consistent cryptographic controls, centralized auditing, and simplified key governance.

An administrator may have permission to view S3 objects without possessing permissions to decrypt them. Similarly, an application may access a bucket while still being denied access to the underlying encryption keys. This separation significantly improves the overall security posture.

The organizational knowledge repository is one of the most sensitive components of the platform. Exposure of stored assets could provide an attacker with detailed knowledge of organizational infrastructure and operational procedures.

6.1 Storage Layer — Amazon S3

Bucket Name	infraguid-knowledge-repository
Access Control	Private Bucket — Public Access Fully Blocked
Versioning	Enabled — supports rollback and audit history
Encryption	SSE-KMS using alias/infraguid-prod-knowledge-key
Logging	CloudTrail and S3 Access Logging both enabled

6.2 Access Layer — Authorized Roles Only

Only designated application roles may access stored documents. Human access is restricted to approved administrative personnel.

- InfraGuidDocumentProcessingRole — Document ingestion and processing
- InfraGuidAgentRuntimeRole — Query-time document retrieval

6.3 Document Ingestion Security Workflow

The document ingestion pipeline is intentionally designed to ensure that only authorized systems can access source documents.

1. Administrator uploads document to the secured ingestion channel.
2. Document is stored in the encrypted S3 bucket immediately upon upload.
3. Document Processing Service assumes the designated IAM role.
4. IAM role requests the encrypted object from S3.
5. S3 requests a decryption operation from AWS KMS.
6. KMS validates the IAM role's permissions against the key policy.
7. Decrypted document is returned exclusively to the processing service.
8. Text extraction and chunking are performed in memory.
9. Amazon Titan Embeddings are generated via Amazon Bedrock.
10. Vector representations are stored in ChromaDB on encrypted EFS.

Throughout this workflow the application never directly handles encryption keys. Key management remains fully delegated to AWS KMS.

InfraGuid follows a strict zero-hardcoded-secrets policy. All credentials, API keys, and sensitive configuration values are stored within AWS Secrets Manager and retrieved dynamically at application startup.

7.1 Policy Enforcement

- No secrets committed to source code repositories.
- No secrets embedded within Docker images or container definitions.
- No secrets stored within Terraform state files or variable definitions.
- No secrets exposed through application logs or error traces.

7.2 Managed Secrets

- MongoDB connection strings
- JWT signing secrets
- Administrative API keys
- Internal service credentials
- Third-party integration tokens

The platform follows the principle of least privilege. Every workload receives only the permissions required to perform its intended function. This limits lateral movement opportunities in the event of a compromise.

Role	Allowed Operations	Denied Operations
InfraGuidAgentRuntimeRole	Query Bedrock, retrieve ChromaDB content, access DynamoDB sessions, read organizational documents	Modify KMS policies, create secrets, delete documents, assume other roles
InfraGuidDocumentProcessingRole	Read uploaded documents, generate embeddings via Bedrock, store vector data in ChromaDB	Access administrative secrets, modify IAM policies, access unrelated resources

Although ChromaDB stores vectorized representations of organizational documents, these vectors still represent sensitive intellectual property. Vector data is therefore protected both logically and physically.

- ChromaDB resides exclusively in private VPC subnets.
- ChromaDB is inaccessible from the public internet.
- Persistent storage resides on Amazon EFS.
- EFS encryption is protected by AWS KMS CMK.
- Access is restricted to backend application instances only.

Every privileged operation within InfraGuid must be fully auditable. AWS CloudTrail provides a complete audit trail of all API activity, key usage, and resource access.

10.1 Audited Event Categories

- KMS decryption and key usage operations
- Secret retrieval events from Secrets Manager
- IAM role assumption and permission escalation
- S3 object access — reads, writes, and deletes
- DynamoDB read and write activity

10.2 Audit Visibility

Audit logs provide complete visibility into who accessed data, when access occurred, which resource was accessed, and which permissions were exercised. This capability is critical during security investigations, compliance reviews, and incident response.

10.3 Supporting Monitoring Services

- Amazon CloudWatch — Real-time metrics, alarms, and dashboards
- S3 Access Logs — Per-request object access records
- AWS Security Hub — Aggregated security findings and insights
- AWS Config — Continuous compliance rule evaluation